

Integración de un Sistema de Control de Versiones en un Sistema Windows

ACTIVIDAD 2

Francisco Javier Otero Herrero
Grupo ATU | 17 de Junio de 2025

Contenido

Práctica	3
Preguntas	3
Pregunta 1.	3
Información adicional para la realización de la Actividad	3
Crear una carpeta para el repositorio.....	4
Descargar e instalar GIT for Windows.....	4
Descargar e instalar WinMerge	8
Configurar GIT.....	10
Comparación de Archivos con WinMerge.....	14

Práctica

Vamos a imponer un sistema para un equipo de desarrollo de forma que no se pierdan las modificaciones que se realicen en el código y permita a varias personas trabajar de forma simultánea en un mismo proyecto.

Preguntas

Pregunta 1.

Realiza la integración de un sistema de control de versiones en un Sistema Operativo Servidor. Los programas a instalar serán:

- *El software de servidor Git o Subversión.*
- *El software gráfico de cliente para controlador de versiones.*
- *El software de comparación de archivos o versiones WinMerge.*

Se debe realizar la instalación y configuración de los programas además del protocolo, servidor, servicio, usuarios, etc., y probar a crear un repositorio para agregar archivos.

Para comprobar la realización correcta de la actividad será necesario que se adjunte una captura de pantalla (junto con la definición) de todos aquellos pasos que se van realizando a lo largo de la actividad.

Información adicional para la realización de la Actividad

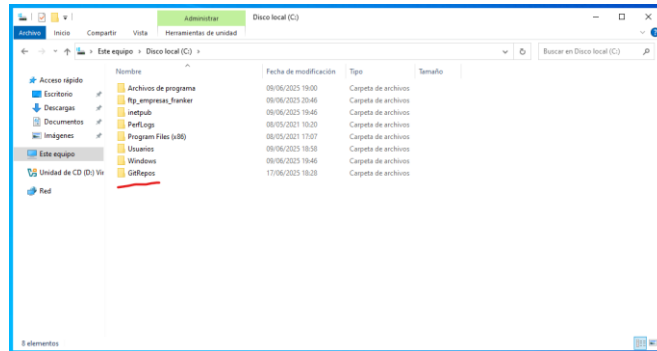
Para la instalación se necesita lo siguiente:

- *Un Sistema Operativo Servidor.*
- *Una carpeta en el servidor con espacio suficiente para albergar los archivos del repositorio*

Integración de un Sistema de Control de Versiones en un Sistema Windows

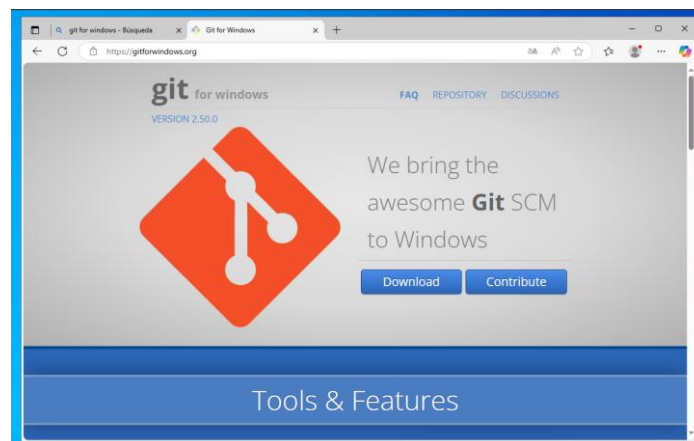
➤ Crear una carpeta para el repositorio

En este apartado vamos a crear una carpeta para el repositorio, para ello en nuestro servidos creamos un nuevo directorio en **C:** que llamaremos **GitRepos**.



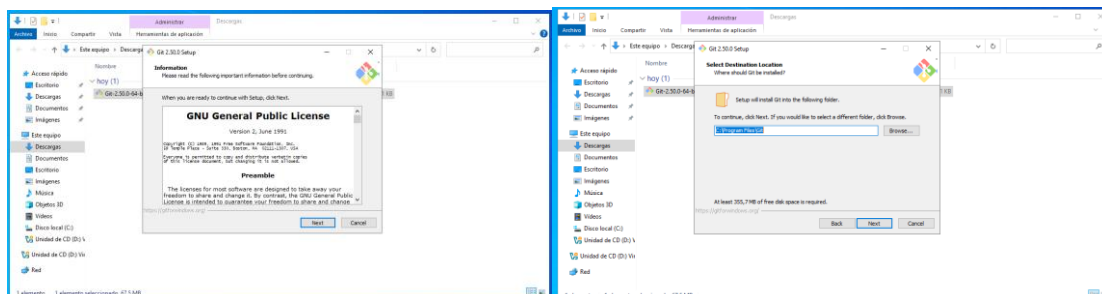
➤ Descargar e instalar GIT for Windows

Git for Windows incluye las herramientas de línea de comandos **Git Bash**, **Git GUI** y otras utilidades necesarias. El primer paso será realizar la descarga desde el sitio oficial:



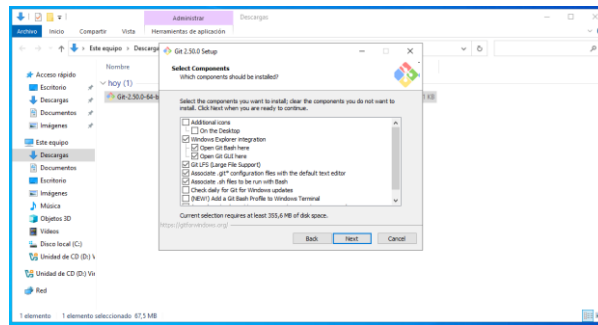
<https://gitforwindows.org/>

Tras la descarga, procedemos a instalar.

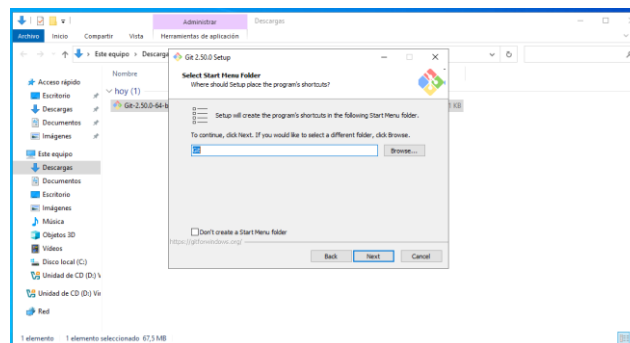


Licencia (GNU General Public License) y directorio donde se instalará.

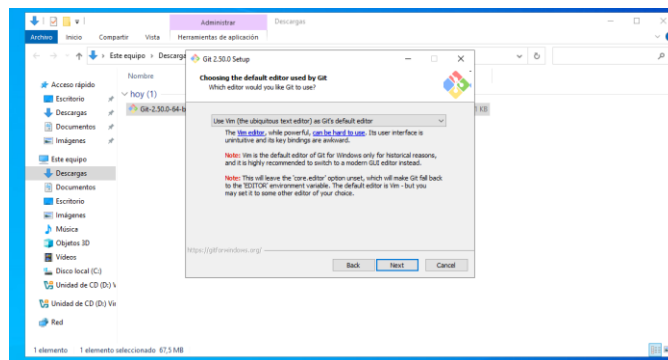
Integración de un Sistema de Control de Versiones en un Sistema Windows



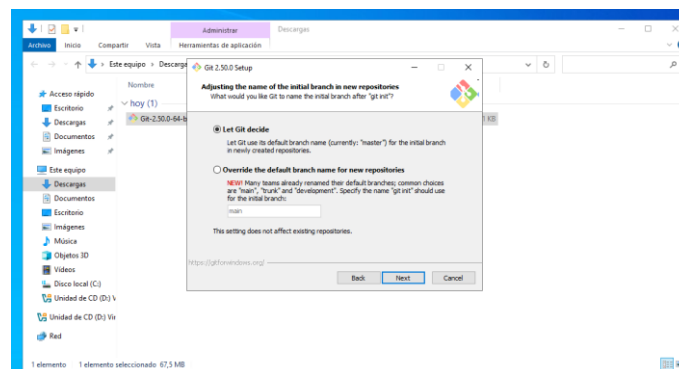
Seleccionamos los componentes, en mi caso he dejado los que venían por defecto. Incluyendo **Git Bash** y **Git GUI** sería suficiente para esta práctica.



Start Menu Folder (Carpeta del Menú de Inicio)



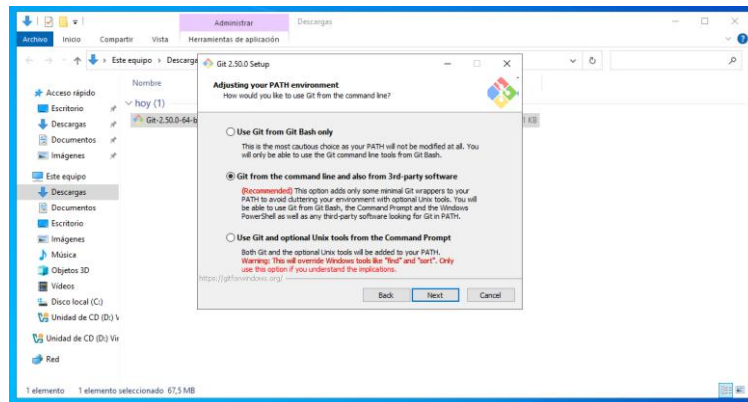
Elegir el editor por defecto usado por Git



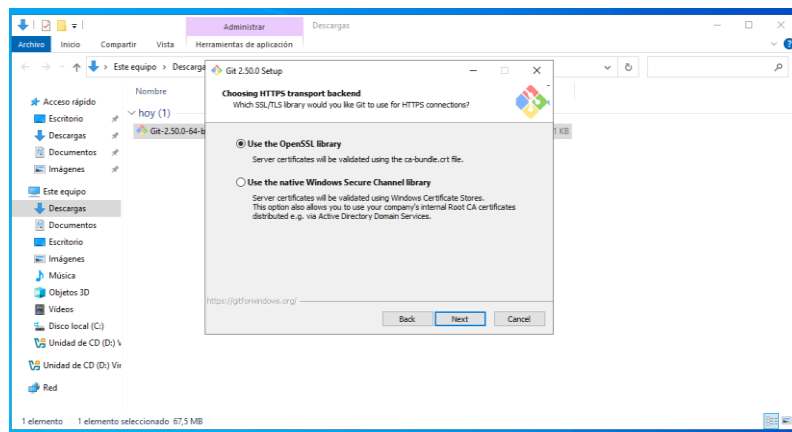
Ajustar el nombre de la rama inicial. Dejamos la opción por defecto (Let Git decide).

Integración de un Sistema de Control de Versiones en un Sistema Windows

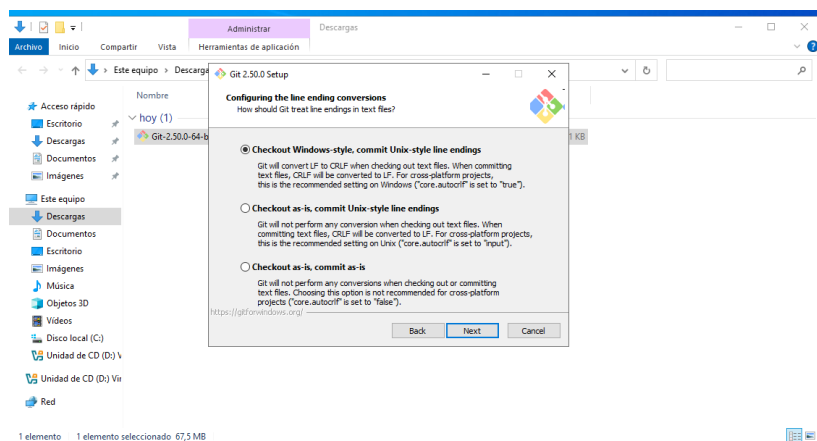
Ajustar nuestro entorno PATH. Elegiremos la segunda opción: ***Git from the command line and also from 3rd-party software*** (Git desde la línea de comandos y también desde software de terceros). Esta opción permite usar Git desde el Símbolo del sistema de Windows y otras herramientas.



Choose HTTPS transport backend (Elegir backend de transporte HTTPS): Elegimos la opción ***Use the OpenSSL library***.

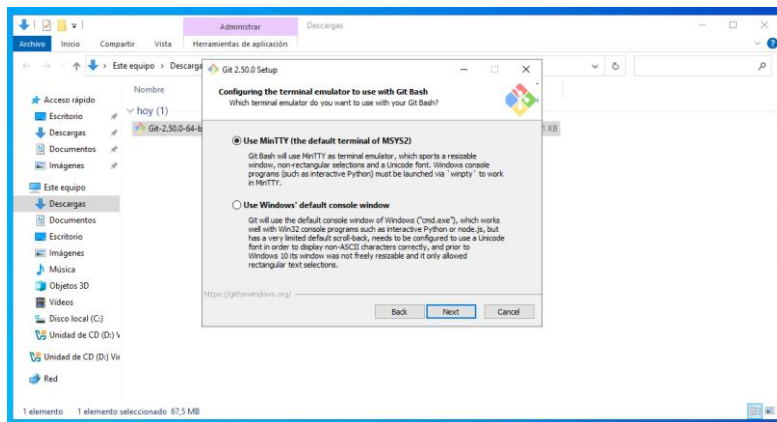


Configure the line ending conversions (Configurar conversiones de fin de línea): Deja la opción por defecto ***Checkout Windows-style, commit Unix-style line endings***. Esto es importante para compatibilidad entre sistemas operativos.

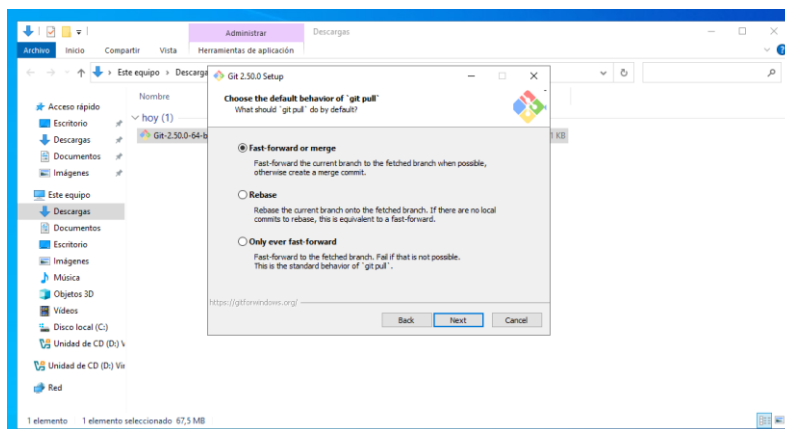


Integración de un Sistema de Control de Versiones en un Sistema Windows

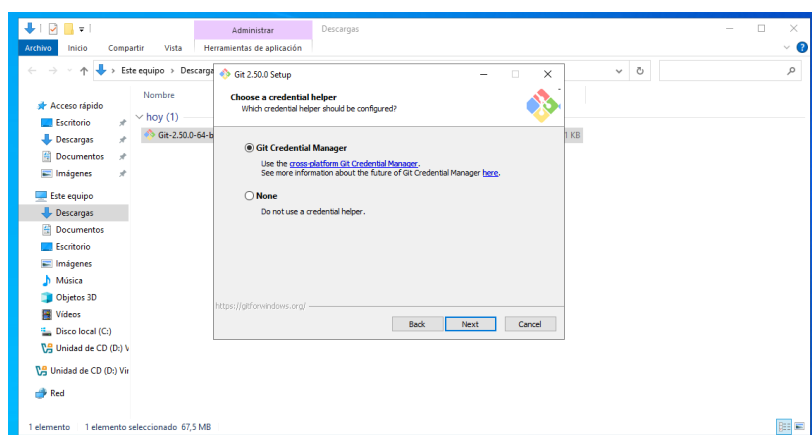
Configure the terminal emulator to use with Git Bash (Configurar el emulador de terminal para usar con Git Bash): Dejamos la opción por defecto **Use MinTTY (the default terminal of MSYS2)**.



Choose the default behavior of git pull (Elegir el comportamiento por defecto de git pull): Dejamos la opción por defecto **Default (fast-forward or merge)**.

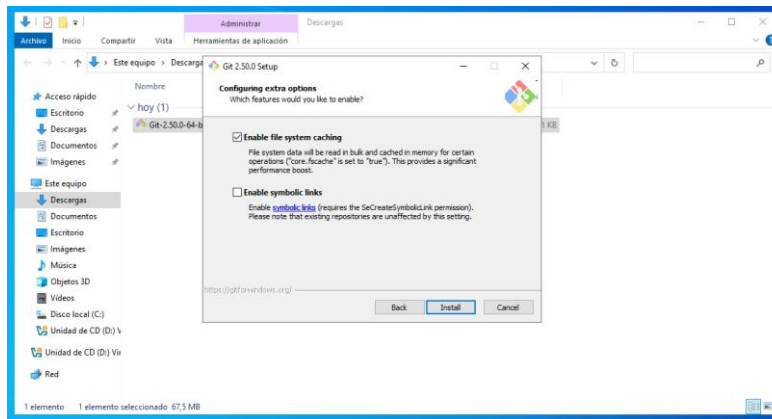


Choose a credential helper (Elegir un asistente de credenciales): Dejar la opción por defecto **Git Credential Manager**.

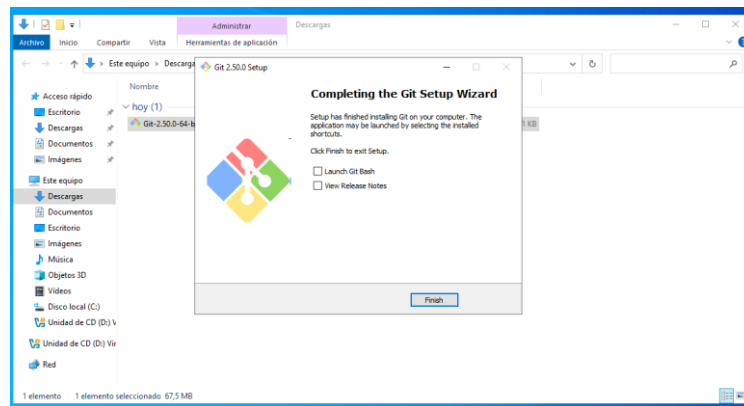


Integración de un Sistema de Control de Versiones en un Sistema Windows

Configure extra options (Configurar opciones extra): Deja las opciones por defecto (*Enable file system caching, Enable symbolic links*).

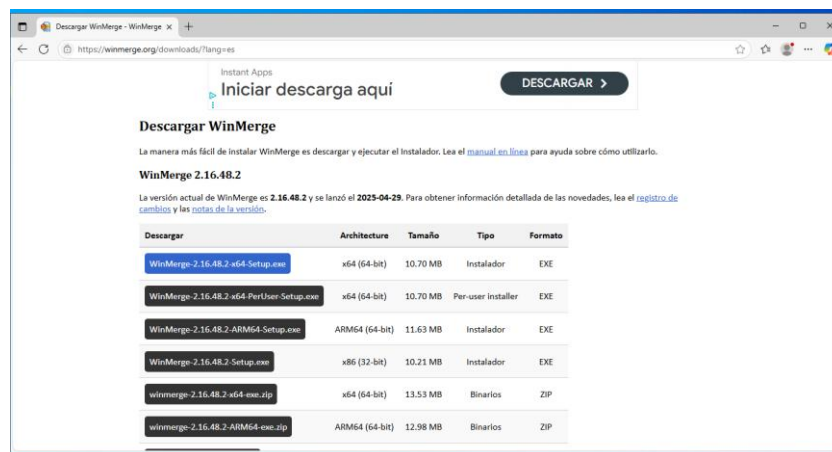


Llegados a este punto, habríamos finalizado la instalación.



➤ **Descargar e instalar WinMerge**

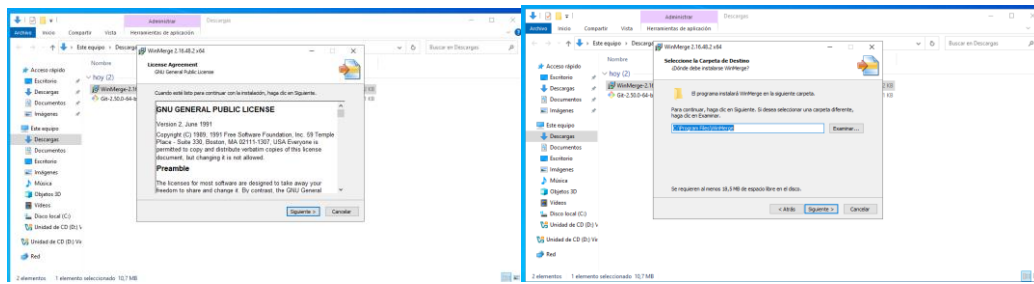
El primer paso será descargar esta herramienta desde el sitio oficial:



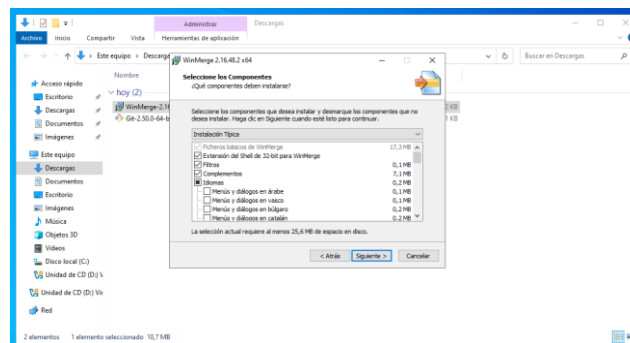
<https://winmerge.org/downloads/?lang=es>

Integración de un Sistema de Control de Versiones en un Sistema Windows

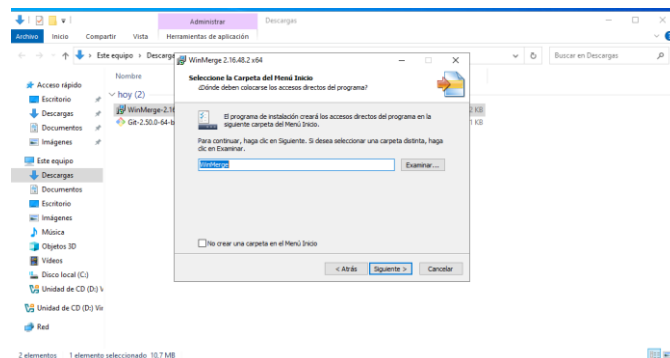
Ejecutamos el instalador.



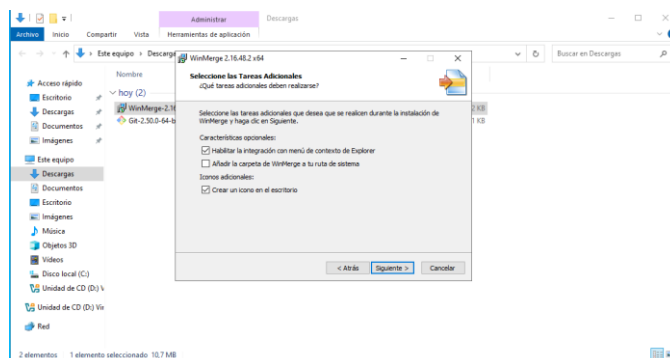
Siguiente y elegimos carpeta de destino. El siguiente paso es elegir el idioma, en nuestro caso español.



Seleccionar carpeta del Menu de Inicio.



Opciones adicionales, dejamos las de por defecto.

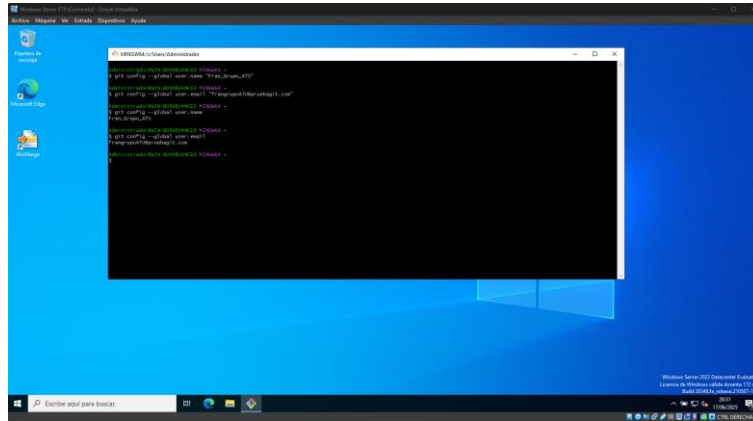


Y con esto ya tendríamos esta herramienta instalada en el servidor.

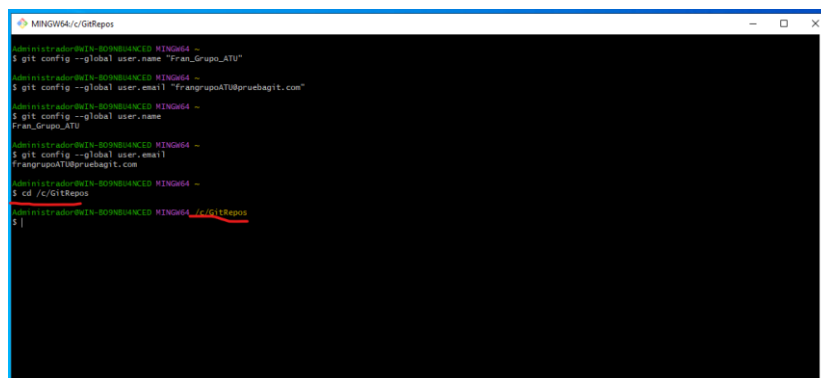
Integración de un Sistema de Control de Versiones en un Sistema Windows

➤ Configurar GIT

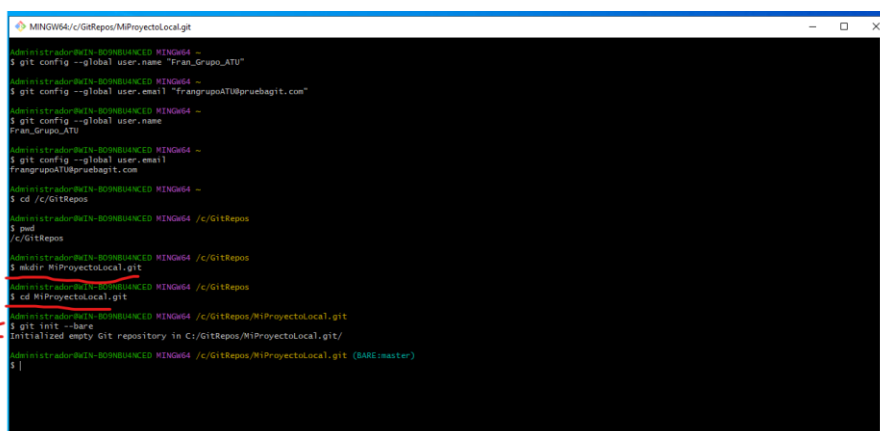
Buscamos **Git Bash** en el menú de inicio de Windows, o hacemos clic derecho en el escritorio y seleccionamos **Git Bash Here**. Configuramos el nombre de usuario y correo electrónico globalmente en Git, y verificamos que se configuraron correctamente:



Navegamos a la carpeta de repositorios que creamos anteriormente.



Crearemos un nuevo repositorio Git vacío (**bare repository**) que actuará como nuestro servidor central. Dentro de GitRepos, creamos un nuevo directorio para el proyecto y lo convertimos en un **repositorio bare**.



Integración de un Sistema de Control de Versiones en un Sistema Windows

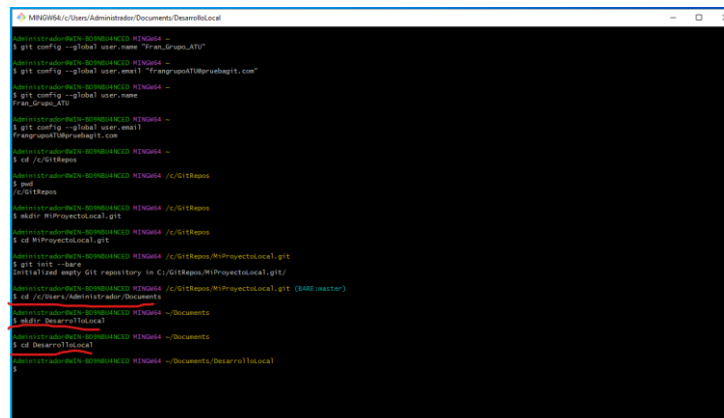
Los comandos utilizados en el anterior paso fueron:

```
mkdir MiProyectoLocal.git → Crear directorio  
cd MiProyectoLocal.git → Movernos dentro del directorio  
git init -bare → Inicializar el repositorio
```

Si todo ha ido bien, veremos un mensaje como **Initialized empty Git repository in...**

Ahora vamos a clonar el repositorio (Actuando como cliente dentro de la misma VM):

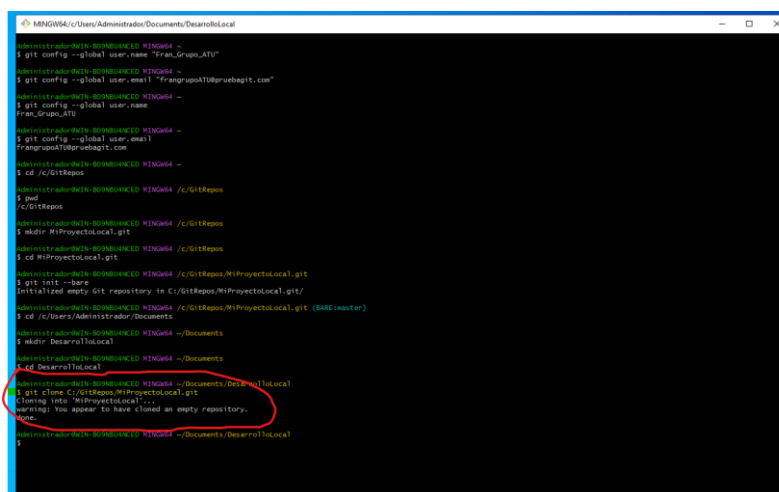
Salimos de la carpeta del repositorio servidor y crearemos una carpeta de trabajo donde simularemos que un desarrollador clona el proyecto para trabajar.



```
Administrator@WIN-80W4NCE3 KINGMA -  
$ git config --global user.name "Fran_Grupo_ATU"  
Administrator@WIN-80W4NCE3 KINGMA -  
$ git config --global user.email "frangrupa@lupruebasgit.com"  
Administrator@WIN-80W4NCE3 KINGMA -  
$ git config --global user.name  
Fran_Grupo_ATU  
Administrator@WIN-80W4NCE3 KINGMA -  
$ git config --global user.email  
frangrupa@lupruebasgit.com  
Administrator@WIN-80W4NCE3 KINGMA -  
$ cd /c/GitRepos  
Administrator@WIN-80W4NCE3 KINGMA /c/GitRepos  
$ pwd  
/c/GitRepos  
Administrator@WIN-80W4NCE3 KINGMA /c/GitRepos  
$ mkdir MiProyectoLocal.git  
Administrator@WIN-80W4NCE3 KINGMA /c/GitRepos  
$ cd MiProyectoLocal.git  
Administrator@WIN-80W4NCE3 KINGMA /c/GitRepos/MiProyectoLocal.git  
$ git init -bare  
Initialized empty Git repository in C:/GitRepos/MiProyectoLocal.git/  
Administrator@WIN-80W4NCE3 KINGMA /c/GitRepos/MiProyectoLocal.git (bare:master)  
$ cd /c/Users/Administrador/Documents  
Administrator@WIN-80W4NCE3 KINGMA ~\Documents  
$ mkdir DesarrolloLocal  
Administrator@WIN-80W4NCE3 KINGMA ~\Documents  
$ cd DesarrolloLocal  
Administrator@WIN-80W4NCE3 KINGMA ~\Documents\DesarrolloLocal  
$
```

En mi caso he creado el directorio en la ruta **C:/Users/Administrador/Documents**.

En este punto, debemos clonar el repositorio bare que hemos creado. La ruta debe ser la ruta física local dentro de la VM.



```
Administrator@WIN-80W4NCE3 KINGMA ~\Documents  
$ cd DesarrolloLocal  
Administrator@WIN-80W4NCE3 KINGMA ~\Documents\DesarrolloLocal  
$ git clone C:/GitRepos/MiProyectoLocal.git  
Cloning into 'MiProyectoLocal'...  
warning: You appear to have cloned an empty repository.  
done.  
Administrator@WIN-80W4NCE3 KINGMA ~\Documents\DesarrolloLocal  
$
```

El comando utilizado para clonar el repositorio ha sido **git clone + ruta del repositorio**. Si todo ha ido bien veremos un mensaje como **Cloning into MiProyectoLocal...** y luego **warning: You appear to have cloned an empty repository.** (Es normal, ya que está vacío).

Integración de un Sistema de Control de Versiones en un Sistema Windows

Verificamos el origen remoto (que apunta al **repositorio local bare**) mediante el comando `git remote -v`, desde dentro de la carpeta del proyecto clonado.

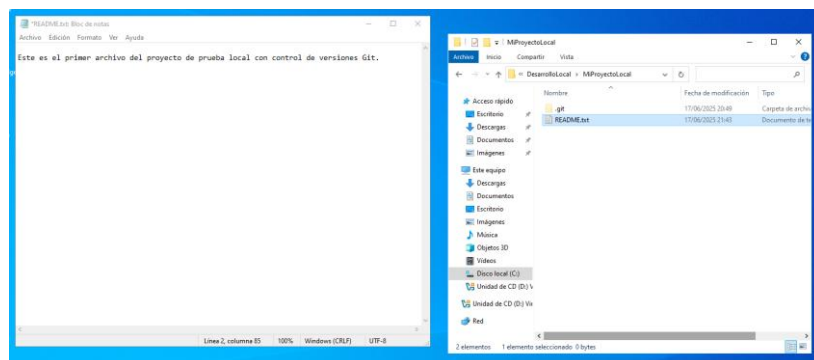
```
MINGW64 ~/Documents/DesarrolloLocal/MiProyectoLocal
$ git config --global user.name
Fran_Grupo_123
$ git config --global user.email
FranGrupo123@protonmail.com
$ cd /c:/GitRepos
$ cd /c:/GitRepos
$ cd /c:/GitRepos
$ mkdir MiProyectoLocal.git
$ cd MiProyectoLocal.git
$ git init -bare
Initialized empty Git repository in C:/GitRepos/MiProyectoLocal.git/
$ cd /c:/Users/Administrador/Documents
$ mkdir DesarrolloLocal
$ cd DesarrolloLocal
$ git clone C:/GitRepos/MiProyectoLocal.git
Cloning into 'MiProyectoLocal'...
warning: You appear to have cloned an empty repository.
done.
$ git remote -v
fatal: not a git repository (or any of the parent directories): .git
$ cd MiProyectoLocal
$ git remote -v
origin C:/GitRepos/MiProyectoLocal.git (fetch)
origin C:/GitRepos/MiProyectoLocal.git (push)
$ git remote -v
origin C:/GitRepos/MiProyectoLocal.git (fetch)
origin C:/GitRepos/MiProyectoLocal.git (push)
```

Si todo ha ido bien, deberíamos ver algo como:

origin C:/GitRepos/MiProyectoLocal.git (fetch)
origin C:/GitRepos/MiProyectoLocal.git (push).

Esto confirma que estás apuntando al repositorio local que hace de servidor.

Es momento de **crear y agregar archivos al repositorio**:



Creamos un documento de texto plano como se puede ver en la imagen anterior. Podemos comprobar desde Git Bash que dicho fichero se ha creado.

```
MINGW64 ~/Documents/DesarrolloLocal/MiProyectoLocal
$ git config --global user.name
Fran_Grupo_123
$ git config --global user.email
FranGrupo123@protonmail.com
$ cd /c:/GitRepos
$ cd /c:/GitRepos
$ cd /c:/GitRepos
$ mkdir MiProyectoLocal.git
$ cd MiProyectoLocal.git
$ git init -bare
Initialized empty Git repository in C:/GitRepos/MiProyectoLocal.git/
$ cd /c:/Users/Administrador/Documents
$ mkdir DesarrolloLocal
$ cd DesarrolloLocal
$ git clone C:/GitRepos/MiProyectoLocal.git
Cloning into 'MiProyectoLocal'...
warning: You appear to have cloned an empty repository.
done.
$ git remote -v
fatal: not a git repository (or any of the parent directories): .git
$ cd MiProyectoLocal
$ git remote -v
origin C:/GitRepos/MiProyectoLocal.git (fetch)
origin C:/GitRepos/MiProyectoLocal.git (push)
$ git remote -v
origin C:/GitRepos/MiProyectoLocal.git (fetch)
origin C:/GitRepos/MiProyectoLocal.git (push)
```

Integración de un Sistema de Control de Versiones en un Sistema Windows

Ahora vamos a agregar el fichero que hemos creado para su seguimiento.

[illegible]

Con el comando ***git add nombredelfichero***, conseguimos que se realice un seguimiento del fichero por si se hiciese alguna modificación. Mediante el comando ***git status***, comprobamos el estado en el que se encuentra.

El siguiente paso será registrar los cambios mediante el comando **git commit -m**.

[illegible]

El último paso será el de **enviar los cambios al repositorio servidor (Push) local**. Los cambios están ahora en nuestro repositorio local (DesarrolloLocal/MiProyectoLocal). Para que estén disponibles en el repositorio central (MiProyectoLocal.git bare en C:\GitRepos), debemos enviarlos.

[illegible]

Veremos mensajes indicando que los objetos han sido escritos al repositorio remoto localmente.

➤ Comparación de Archivos con WinMerge

Ahora que tenemos un archivo en el repositorio y WinMerge instalado, vamos a modificarlo y usar WinMerge para ver las diferencias. Abrimos el fichero y añadimos una nueva línea de texto.

[illegible]

Mediante el comando **git status** podemos comprobar que el fichero ha sido modificado. Si ejecutamos **git diff** mostrará las diferencias del fichero directamente en la terminal.

```

C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal> git branch --set-upstream-to=origin/master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal> git add README.txt
[WPProjectLocal:master] MINGW64 ~\Documents\DesarrolloLocal\WPProjectLocal (master)
$ git add README.txt

C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal> git commit -m "Commit inicial del repositorio local"
[WPProjectLocal:master] MINGW64 ~\Documents\DesarrolloLocal\WPProjectLocal (master)
$ git commit -m "Commit inicial del repositorio local"
$ file changed, 2 insertions(+)
```

Creando el master remoto

```

C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal> git push --set-upstream origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 164 bytes | 86.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), push-reused 0 (from 0)
To C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal:master
   1c19e68..1704688 master -> master

C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal> git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)

nothing to commit, working tree clean

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal> git diff
diff --git a/README.txt b/README.txt
new file mode 100644
index 0000000..1c19e68
++ b/README.txt
```

Verificando el estado del proyecto de grado local con control de versiones git.
no me incluye el end of file

Verificando el estado del proyecto de grado local con control de versiones git.
no me incluye el end of file

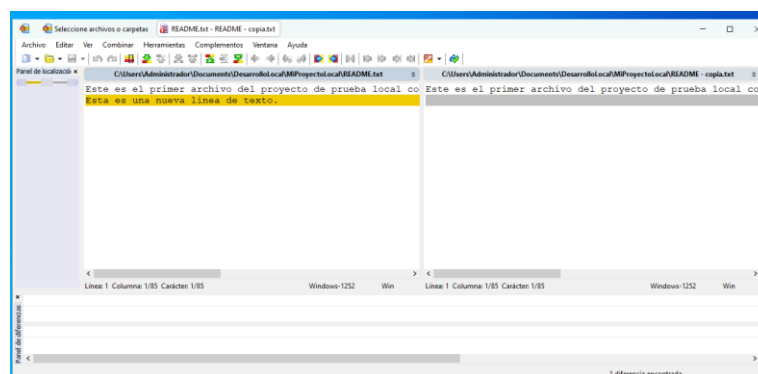
```

C:\Users\alejandro\Documents\Proyectos\GitHub\WPProjectLocal> git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)

nothing to commit, working tree clean
```

Si abrimos la herramienta **WinMerge** que instalamos anteriormente y comparamos dos ficheros.



Para esta comparación se ha creado una copia del fichero README.txt antes de ser modificado.